

Lab2

Name: Kathyayani Choudhari

Roll no : 2303A53001

Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab)and Cursor AI

Task 1: Statistical Summary for Survey Data

❖ Scenario:

You are a data analyst intern working with survey responses stored as numerical lists.

The screenshot displays a Google Colab notebook titled 'Lab2_Task1.ipynb'. The notebook contains a Python function named `statistical_summary` that calculates the mean, minimum, and maximum of a list of numbers. The function includes docstrings for its purpose, arguments, and returns, as well as a sample list of survey responses. The function is executed, and the output is displayed in the terminal.

```
[1] def statistical_summary(data_list):  
    """  
    Calculates and prints the mean, minimum, and maximum of a list of numbers.  
    Args:  
        data_list (list): A list of numerical survey responses.  
    Returns:  
        None: Prints the summary directly.  
    """  
    if not data_list:  
        print("The input list is empty. Cannot calculate summary statistics.")  
        return  
  
    # Calculate mean  
    total_sum = 0  
    for number in data_list:  
        total_sum += number  
    mean_value = total_sum / len(data_list)  
  
    # Calculate minimum and maximum  
    min_value = data_list[0]  
    max_value = data_list[0]  
    for number in data_list:  
        if number < min_value:  
            min_value = number  
        if number > max_value:  
            max_value = number  
  
    print(f"Statistical Summary of Survey Responses:")  
    print(f" Mean (Average): {mean_value:.2f}")
```

The Gemini AI chat interface is visible on the right side of the notebook. It shows a conversation where the user asks for a Python function to calculate the mean, minimum, and maximum of a list of numerical survey responses. The Gemini AI assistant responds with a detailed explanation and a sample list of survey responses, along with a brief explanation of the function output.

What can I help you build?

Gemini 2.5 Flash

The screenshot shows a Jupyter Notebook titled 'Lab2_Task1.ipynb'. The code defines a function `statistical_summary` that takes a list of survey responses and returns a dictionary with mean, minimum, and maximum values. The function is called with a sample list of responses, and the output is displayed as a dictionary.

```
[3]:  
def statistical_summary(data_list):  
    """  
    Checks if a given number is an Armstrong number.  
    An Armstrong number is a number that is equal to the sum of its own  
    digits each raised to the power of digits.  
    Args:  
        number (int): The integer to check.  
    Returns:  
        bool: True if the number is an Armstrong number, False otherwise.  
    """  
    if not isinstance(number, int) or number < 0:  
        print("Please provide a non-negative integer.")  
        return False  
    # Convert the number to a string to easily count digits and iterate through them  
    num_str = str(number)  
    num_digits = len(num_str)  
    sum_of_powers = 0  
    # Iterate through each digit, raise it to the power of num_digits, and add to sum  
    for digit_char in num_str:  
        digit = int(digit_char)  
        sum_of_powers += digit ** num_digits  
    # Compare the sum with the original number  
    return sum_of_powers == number  
    """  
    print("Statistical Summary of Survey Responses:")  
    print(f" Mean (Average): {mean_value:.2f}")  
    print(f" Minimum Value: {min_value}")  
    print(f" Maximum Value: {max_value}")  
    """  
    # Sample list of survey responses  
    sample_responses = [2, 5, 1, 8, 3, 9, 4, 7, 6, 10]  
    # Call the function with the sample list  
    statistical_summary(sample_responses)
```

Statistical Summary of Survey Responses:
Mean (Average): 5.50
Minimum Value: 1
Maximum Value: 10

Code Explanation

The `statistical_summary` function takes a list `data_list` as input. It first checks if the list is empty to prevent errors.

- Mean Calculation:** It iterates through the list to sum all numbers and then divides the total sum by the count of numbers in the list (`len(data_list)`) to find the average.
- Min/Max Calculation:** It initializes `min_value` and `max_value` with the first element of the list. Then, it iterates through the rest of the list, updating `min_value` if a smaller number is found and `max_value` if a larger number is found.
- Output:** Finally, it prints the calculated mean (formatted to two decimal places), minimum, and maximum values in a clear, readable format.

Task 2: Armstrong Number – AI Comparison

❖ Scenario:

You are evaluating AI tools for numeric validation logic.

Google Gemini (Colab) output:-

The screenshot shows a Jupyter Notebook titled 'Lab2_Task2.ipynb'. The code defines a function `is_armstrong` that takes an integer and returns a boolean indicating if it is an Armstrong number. The function is called with the sample number 153, and the output is displayed as a boolean.

```
[4]:  
def is_armstrong(number):  
    """  
    Checks if a given number is an Armstrong number.  
    An Armstrong number is a number that is equal to the sum of its own  
    digits each raised to the power of digits.  
    Args:  
        number (int): The integer to check.  
    Returns:  
        bool: True if the number is an Armstrong number, False otherwise.  
    """  
    if not isinstance(number, int) or number < 0:  
        print("Please provide a non-negative integer.")  
        return False  
    # Convert the number to a string to easily count digits and iterate through them  
    num_str = str(number)  
    num_digits = len(num_str)  
    sum_of_powers = 0  
    # Iterate through each digit, raise it to the power of num_digits, and add to sum  
    for digit_char in num_str:  
        digit = int(digit_char)  
        sum_of_powers += digit ** num_digits  
    # Compare the sum with the original number  
    return sum_of_powers == number  
    """  
    # Sample number to test  
    sample_number = 153 # Example of an Armstrong number
```

is_armstrong(153) is True

Google Gemini output:

You are an AI coding assistant. Write a Python program to check whether a given number is an Armstrong number. Requirements: - Define a function named `is_armstrong(number)` - Do not use external libraries - Clearly explain the logic step by step - Take a sample number as input - Print whether the number is an Armstrong number or not - Keep the code simple and beginner-friendly After the code, provide a short explanation of how the Armstrong check works.

Certainly! Here's a Python program to check whether a given number is an Armstrong number, along with a step-by-step explanation.

The `is_armstrong` function successfully identified that 153 is indeed an Armstrong number, as shown by the output: "The number 153 IS an Armstrong number."

Test 'is_armstrong' with other sample numbers
Find Armstrong numbers within a specified range
Explain the code for 'statistical_summary' function

What can I help you build?

Gemini 2.5 Flash

The screenshot shows a Jupyter Notebook titled "Lab2_Task2ipynb". The code cell contains a Python script that defines a function `is_armstrong` and tests it with several numbers. The output shows that 153 is an Armstrong number, while 123 is not.

```
[4] ✓ Os
# Sample number to test
sample_number = 153 # Example of an Armstrong number
# sample_number = 9 # Example of an Armstrong number
# sample_number = 370 # Example of an Armstrong number
# sample_number = 1634 # Example of an Armstrong number
# sample_number = 123 # Example of a non-Armstrong number

if is_armstrong(sample_number):
    print(f"The number {sample_number} IS an Armstrong number.")
else:
    print(f"The number {sample_number} IS NOT an Armstrong number.")

... The number 153 IS an Armstrong number.
```

How the Armstrong Check Works

- Count Digits:** The first step is to determine the number of digits in the given number. This is done by converting the number to a string and getting its length.
- Initialize Sum:** A variable `sum_of_powers` is initialized to zero. This variable will accumulate the sum of each digit raised to the power of the total number of digits.
- Iterate and Calculate Power:** The code then iterates through each digit of the number (again, by treating it as a string). For each digit:
 - It converts the digit character back into an integer.
 - It raises this integer digit to the power of `num_digits` (the total count of digits).
 - The result is added to `sum_of_powers`.
- Compare:** Finally, the calculated `sum_of_powers` is compared with the original `number`. If they are equal, the number is an Armstrong number; otherwise, it is not.

GitHub Copilot:-

The screenshot shows a VS Code editor with a file named `Armstrong_num.py`. The code defines a function `is_armstrong` and tests it with 153 and 123. The terminal output shows the results of the function calls.

```
1 # Write a Python function to check whether a number is an Armstrong number.
2 # The function should:
3 # - Accept an integer as input
4 # - Calculate the sum of digits raised to the power of number of digits
5 # - Return True if it is an Armstrong number, otherwise False
6 # - Include a sample input and output
7 def is_armstrong_number(n):
8     num_str = str(n)
9     num_digits = len(num_str)
10    sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)
11    return sum_of_powers == n
12
13 # Sample input and output
14 print(is_armstrong_number(153)) # Output: True
15 print(is_armstrong_number(123)) # Output: False
16
```

Terminal Output:

```
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe c:/Users/sonu/Desktop/AI_Coding/Armstrong_num.py
True
False
PS C:\Users\sonu\Desktop\AI_Coding>
```

Comparison table of Google Gemini v/s GitHub Copilot:-

Criteria	Google Gemini (Colab)	GitHub Copilot (VS Code)
Code Readability	Very high readability with detailed comments, docstring, and descriptive variable names. Suitable for academic submissions.	Concise and clean code with fewer comments. Easy for experienced developers but slightly less descriptive for beginners.
Logic Clarity	Logic is written step-by-step with explicit loops and intermediate variables, making the Armstrong concept very clear.	Logic is written step-by-step with explicit loops and intermediate variables, making the Armstrong concept very clear.
Use of Functions	Uses a well-documented function <code>is_armstrong(number)</code> with input validation and return value.	Uses a simple function <code>is_armstrong_number(n)</code> without input validation but function usage is correct and efficient.
Explanation Quality	Excellent explanation provided after the code, including digit counting, power calculation, and comparison steps	Excellent explanation provided after the code, including digit counting, power calculation, and comparison steps
Beginner Friendliness	Highly beginner-friendly due to verbose explanation, comments, and sample outputs shown clearly.	More suitable for intermediate users; beginners may need additional explanation.

Task 3: Leap Year Validation Using Cursor AI

◆ Scenario:

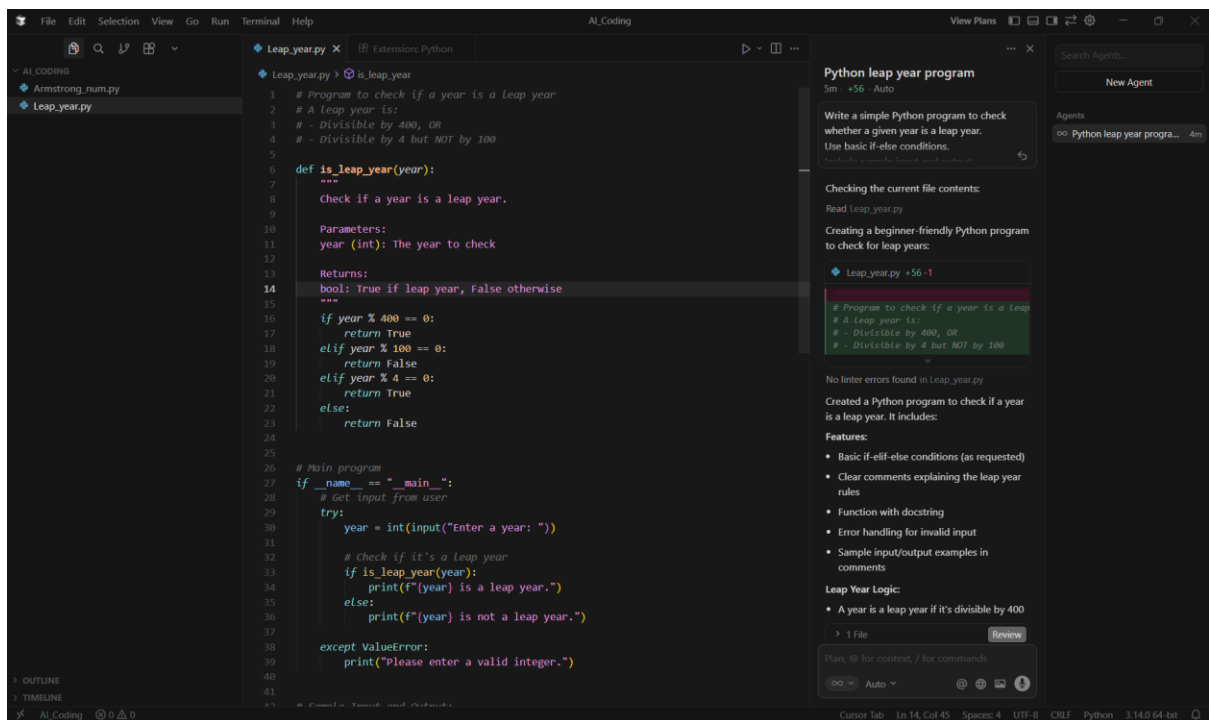
You are validating a calendar module for a backend system.

Prompt: - Write a simple Python program to check whether a given year is a leap year.

Use basic if-else conditions.

Include sample input and output.

Keep the code beginner-friendly.



The screenshot displays the Cursor AI IDE interface. The main editor window shows a Python file named `Leap_year.py` with the following code:

```
1 # Program to check if a year is a leap year
2 # A leap year is:
3 # - Divisible by 400, OR
4 # - Divisible by 4 but NOT by 100
5
6 def is_leap_year(year):
7     """
8     Check if a year is a leap year.
9
10    Parameters:
11    year (int): The year to check
12
13    Returns:
14    bool: True if leap year, False otherwise
15    """
16    if year % 400 == 0:
17        return True
18    elif year % 100 == 0:
19        return False
20    elif year % 4 == 0:
21        return True
22    else:
23        return False
24
25
26 # Main program
27 if __name__ == "__main__":
28     # Get input from user
29     try:
30         year = int(input("Enter a year: "))
31
32         # Check if it's a leap year
33         if is_leap_year(year):
34             print(f"{year} is a leap year.")
35         else:
36             print(f"{year} is not a leap year.")
37
38     except ValueError:
39         print("Please enter a valid integer.")
40
41
```

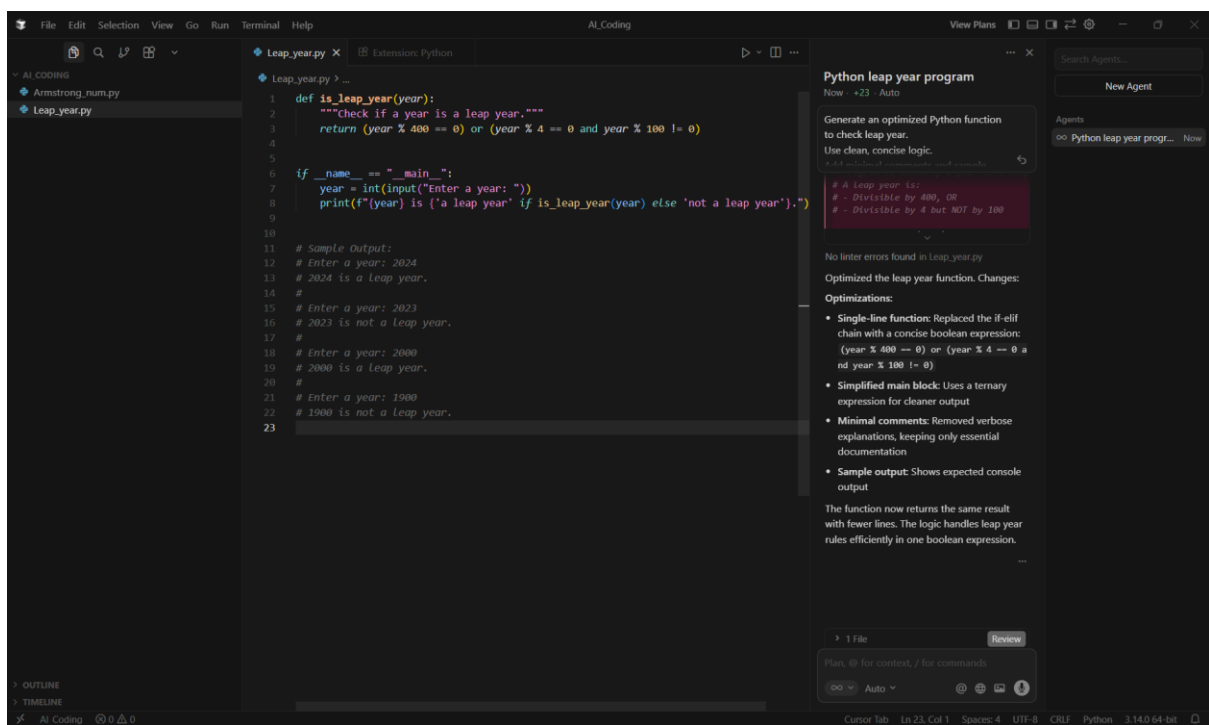
The right sidebar shows the AI agent's response to the prompt. It includes a summary of the task, a list of features (Basic if-else conditions, Clear comments, Function with docstring, Error handling, Sample input/output), and a list of leap year logic rules. The bottom status bar indicates the current file is `Leap_year.py` at line 14, column 45, with a Python 3.14.0 64-bit interpreter.

```
Problems Output Debug Console Terminal Ports Python + - [ ] [X] ... ^ X
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe c:/Users/sonu/Desktop/AI_Coding/L
eap_year.py
Enter a year: 2004
2004 is a leap year.
PS C:\Users\sonu\Desktop\AI_Coding> |
```

Prompt 2 : - Generate an optimized Python function to check leap year.

Use clean, concise logic.

Add minimal comments and sample output.



```
Problems Output Debug Console Terminal Ports Python + - [ ] [X] ... ^ X
PS C:\Users\sonu\Desktop\AI_Coding> & C:/Python314/python.exe c:/Users/sonu/Desktop/AI_Coding/Leap_year.py
Enter a year: 2005
2005 is not a leap year.
PS C:\Users\sonu\Desktop\AI_Coding> |
```

Ctrl+K to generate command

Aspect	Version 1	Version 2
Code Style	Step-by-step if-else	One-line logical expression
Readability	Very high (easy for beginners)	Moderate (needs logic understanding)
Length	Longer	Short and optimized
Beginner Friendliness	Excellent	Medium
Efficiency	Normal	More efficient and clean

Task 4: Student Logic + AI Refactoring (Odd/Even Sum)

❖ Scenario:

Company policy requires developers to write logic before using AI.

